



OPTATIVA Machine Learning

Autor/a: Núria Pujol Vilanova | Curs: 2025-2026

Institut TIC Barcelona

Estructures de control condicionals

Les estructures de control són el conjunt general d'instruccions que determinen l'ordre en què s'executen les diferents parts del codi.

Inclouen la **seqüencial** (instruccions una darrere l'altra), la **condicional** (executar instruccions segons condicions) i la **iterativa** (repetir instruccions).

Les estructures condicionals permeten executar o no un bloc de codi depenent de si es compleix una condició. En funció del resultat de la condició, les estructures condicionals **bifurquen el flux d'execució** (el programa segueixi diferents camins).

Operadors de comparació

Els operadors de comparació permeten comparar valors numèrics, cadenes o altres objectes, retornant un valor booleà (`True` o `False`).

Són essencials en **condicions i bucles**.

Python disposa dels següents operadors de comparació:

Operador	Significat	Exemple	Resultat
<code>==</code>	Igual a	<code>a == b</code>	Retorna True si a és igual a b
<code>!=</code>	Diferent de	<code>a != b</code>	Retorna True si a i b són diferents
<code><</code>	Menor que	<code>a < b</code>	Retorna True si a és menor que b
<code><=</code>	Menor o igual que	<code>a <= b</code>	Retorna True si a és menor o igual que b
<code>></code>	Major que	<code>a > b</code>	Retorna True si a és major que b
<code>>=</code>	Major o igual que	<code>a >= b</code>	Retorna True si a és major o igual que b

```
a = 10
b = 20
a < b      # True
a == b     # False
a != b     # True
```

És important destacar que comparar valors de tipus incompatibles (un enter amb una cadena, per exemple) pot donar un resultat `False` o una excepció de tipus (`TypeError`).

```
In [ ]: 2 < "2"
```

Operadors booleans

El operadors booleans en Python són:

- **and** retorna **True** si els dos operands són True.
- **or** retorna True si algun operand és True.
- **not** inverteix el valor booleà.

A	B	not A	A and B	A or B
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

```
x = 5
y = 10
(x < 10) and (y > 5)      # True
not (x == 5)              # False
(x > 10) or (y == 10)     # True
```

El operadors lògics s'utilitzen per crear condicions més complexes i controlar el flux del programa.

Python disposa de tres operadors booleans principals: **and**, **or** i **not**. Aquests són els únics operadors lògics (booleans) integrats al llenguatge.

No existeix un operador específic per fer la funció booleana xor (or exclusiu) a Python, però es pot aconseguir el comportament d'un **xor** mitjançant expressions lògiques, o bé utilitzant l'operador bit a bit **^** quan es treballa amb valors booleans.

```
a = True
b = False

# Amb operador ^
print(a ^ b)                # True

# important xor de operator
from operator import xor
print(xor(a, b))            # True

# ús de combinat de and not i or
print((a and not b) or (not a and b)) # True
```

Precedència dels Operadors

Python assigna una precedència als operadors que determina l'ordre en què es calculen les expressions.

Els operadors de comparació (**<**, **>**, **==**, etc.) tenen major precedència que els operadors lògics (**and**, **or**).

Això vol dir que les expressions següents són equivalents:

```
a = 10
b = 20

(a < 20) and (b < 30)    #True

a < 20 and b < 30        #True
```

L'ús dels parèntesis no és estrictament necessari però en millora la claredat i evita confusions. També ajuda a mantenir el codi llegible i a evitar errors.

Sentències Condicionals

Permeten executar un bloc de codi només quan es compleix una condició específica dotant al llenguatge de mecanismes per al control de flux del programa.

El control de flux permet que un programa no només s'executi de manera seqüencial, sinó que pugui prendre diferents rutes, repeteixi certs blocs de codi o decideixi quines instruccions executar depenent de situacions específiques.

Amb les sentències condicionals (`if`, `elif`, `else`), el programa pot prendre decisions, activant o desactivant blocs de codi, en funció del resultat d'una condició booleana.

El codi resultant serà més **dinàmic i adaptable** a diferents situacions: diferents entrades o esdeveniments.

Sentència `if`

La instrucció `if` executa un bloc de codi només si la condició és certa.

```
In [ ]: num = 7
        if num > 0:
            print(f"El número {num} és positiu")
```

Si `num` és més gran que 0, es mostra el missatge, sino el programa "saltarà" aquest bloc del codi.

Condicions aniuades

En Python, és possible aniuar sentències condicionals (col·locar una sentència `if` dins d'una altra per comprovar múltiples condicions).

Una forma més compacta però equivalent seria utilitzar l'operador `and` per combinar les dues condicions en un sol `if`.

Python ofereix també una sintaxi encara més elegant, anomenada "chaining" o encadenament d'operadors de comparació.

```
In [ ]: #num = 7 #definit anteriorment

        #aniuament de conditionals
        if num > 0:
            if num < 10:
                print("El número està entre 0 i 10")

        #aniuament de conditionals amb l'operador and
        if (num > 0) and (num < 10):
```

```
print("El número està entre 0 i 10")

#encadenament d'operadors ("chaining")
if 0 < num < 10:
    print("El número està entre 0 i 10")
```

Sentència else

Permet executar un bloc de codi alternatiu si la condició de l' `if` és `False` .

```
In [ ]: if num % 2 == 0:
        print(f"El número {num} és parell")
        else:
        print(f"El número {num} és senar")
```

Sentencia elif

Només s'executa el primer bloc on la condició és certa, i es salten la resta.

```
In [ ]: day = "Divendres"

if day == "Dilluns":
    print("Treballar")
elif day == "Dimecres":
    print("Reunió")
elif day == "Divendres":
    print("Sortida amb amics")
else:
    print("Cap activitat programada")
```

```
In [ ]: nota = 8.6 # nota de l'alumne

if nota < 5.0:
    print("Has suspès")
elif nota < 6.5:
    print("Has aprovat")
elif nota < 9.0:
    print("Els teus resultats són notables")
else:
    print("Excel·lent feina!")
```

Operador Ternari

L'operador ternari de Python permet assignar valors o executar expressions d'una manera més compacta amb `if-else` en una sola línia.

```
In [ ]: edat = 17
if edat >= 18:
    resultat = "Ets major d'edat"
else:
    resultat = "Ets menor d'edat"
print(resultat) # Sortida: Ets menor d'edat
```

```
In [ ]: resultat_ternari = "Ets major d'edat" if edat >= 18 else "Ets menor d'edat"
print(resultat_ternari) # Sortida: Ets menor d'edat
```

Operador Walrus

Introduït a Python 3.8, l'operador walrus ("morsa") `:=` permet assignar un valor a una variable dins d'una expressió. Obtenim així un codi més net i eficient.

```
In [ ]: #fent ús de l'operador walrus
n = 10
if (doble := n * 2) > 15:
    print(f"El doble de {n} és {doble} i és major que 15")
else:
    print(f"El doble de {n} és {doble} i no és major que 15")

#equivalent sense fer ús de l'operador walrus
n = 10
doble = n * 2
if doble > 15:
    print(f"El doble de {n} és {doble} i és major que 15")
else:
    print(f"El doble de {n} és {doble} i no és major que 15")
```

Aquest operador evita la necessitat de fer l'assignació i la comprovació en dues línies separades.

Operadors d'Identitat

Els operadors d'identitat `is` i `is not` comparen si dues variables apunten exactament al mateix objecte en memòria (més enllà de tenir el mateix valor).

```
In [ ]: a = 5
b = 5
c = 10

print(a is b)      # True, perquè Python reutilitza l'objecte 5 a memòria
print(a is c)      # False, 5 i 10 són objectes diferents
print(a is not c)  # True, perquè no són el mateix objecte
```

```
In [ ]: print(id(a))
print(id(b))
print(id(c))
```

```
In [ ]: x = "hola"
y = "hola"
z = "adéu"

print(x is y)      # True, perquè les cadenes literals iguals comparteixen objecte
print(x is z)      # False
```

```
In [ ]: print(id(x))
print(id(y))
print(id(z))
```

```
In [ ]: paraula1 = "patata"
paraula2 = "patata"

print(paraula1==paraula2)
print(id(paraula1))
print(id(paraula2))

paraula1 = paraula1 + '!'

print(paraula1)
print(paraula1==paraula2)
```

```
print(id(paraula1))
print(id(paraula2))
```

Els operadors d'identitat en Python (`is` i `is not`) tenen un sentit clarament més rellevant quan treballem amb estructures més complexes o quan manipulem objectes mutables o immutables.

Operadors Bit a Bit

Els operadors bit a bit en Python permeten manipular individualment els bits que formen la representació binària dels nombres enters.

Aquests operadors són similars als que es troben en altres llenguatges de programació com C o Java, però tenen un comportament diferent respecte als operadors lògics booleans `and` i `or` .

```
In [ ]: val1 = 10 # binari: 1010
        val2 = 12 # binari: 1100

        print("val1 =", val1, "binari:", bin(val1))
        print("val2 =", val2, "binari:", bin(val2))
```

```
In [ ]: # AND bit a bit
        print("val1 & val2 =", val1 & val2, "binari:", bin(val1 & val2)) # 8 (1000)
```

```
In [ ]: # OR bit a bit
        print("val1 | val2 =", val1 | val2, "binari:", bin(val1 | val2)) # 14 (1110)
```

```
In [ ]: # NOT bit a bit
        print("~val1 =", ~val1, "binari:", bin(~val1)) # -11 (complement a dos)
```

```
In [ ]: # Desplaçament a la dreta
        print("val1 >> 2 =", val1 >> 2, "binari:", bin(val1 >> 2)) # 2 (10)
```

Encara que els operadors bit a bit puguin semblar semblants als operadors lògics `and` i `or` , la seva funció és molt diferent: treballen directament amb la **representació binària** dels enters i permeten manipulacions a molt **baix nivell** , útils en programació de sistemes, encriptació, manipulació de dades, etc.